ffmpeg-micro.com

How to Create a Video from Images with FFmpeg

Javid Jamae

8–10 minutes

Turning a folder of images into a playable video file is one of the most common FFmpeg tasks. Product screenshots for a demo reel, AI-generated frames for a social post, timelapse photos from a construction site. The command is short but the flags trip people up, especially `-pix_fmt`, `-framerate`, and image naming patterns.

FFmpeg can convert a numbered image sequence into an H.264 MP4 with one command. If you need to do this programmatically without installing FFmpeg, the FFmpeg Micro API accepts image URLs directly and returns a finished video file.

The Basic Command: Image Sequence to MP4

If your images are named with sequential numbers (`frame_001.png`, `frame_002.png`, etc.), this is the command:

```
ffmpeg -framerate 1 -i frame_%03d.png -c:v
libx264 -pix_fmt yuv420p -r 30 output.mp4
```

What each flag does:

- `-framerate 1` tells FFmpeg to read one image per second. Each image becomes one second of video. Change this to 2 for half-second images, or `1/5` for five seconds per image.
- `-i frame_%03d.png` is the input pattern. `%03d` matches zero-padded three-digit numbers (001, 002, 003). Use `%04d` for four digits.
- `-c:v libx264` encodes with H.264, the most widely supported video codec.
- `-pix_fmt yuv420p` forces a pixel format compatible with every browser and player. Without this, PNGs with alpha channels produce videos that play on VLC but show a black screen on Chrome, Safari, and most phones.
- `-r 30` sets the output frame rate to 30fps. This doesn't change how long each image shows. It controls how many duplicate frames FFmpeg writes per image, which affects playback smoothness on some players.

  With 5 input images at `-framerate 1`, the output is a 5-second video. At `-framerate 2`, it's 2.5 seconds.

Using Glob Patterns Instead of Numbered Files

Not every image set follows a clean `001, 002, 003` naming pattern. If your files are named `screenshot-monday.png`, `screenshot-tuesday.png`, or just don't have consistent numbering, use a glob:

```
ffmpeg -framerate 1 -pattern_type glob -i
'*.png' -c:v libx264 -pix_fmt yuv420p -r 30
output.mp4
```

FFmpeg sorts glob matches alphabetically, so the image

order depends on your filenames. Rename files with a numeric prefix (`01-screenshot.png`, `02-screenshot.png`) if the order matters.

Note: Glob patterns work on macOS and Linux. On Windows, use the numbered sequence pattern or pipe images through stdin with `-f image2pipe`.

One Image, Specific Duration

Sometimes you just need a single image displayed for N seconds. A title card, a placeholder, or a background for an audio track:

```
ffmpeg -loop 1 -i title-card.png -c:v libx264 -t 10 -pix_fmt yuv420p -r 30 output.mp4
```

`-loop 1` tells FFmpeg to keep reading the same image. Without it, you get a single-frame file regardless of the `-t` value. `-t 10` sets the output duration to 10 seconds.

Controlling Quality with CRF

The default CRF (Constant Rate Factor) for libx264 is 23. Lower values mean higher quality and larger files. Higher values mean smaller files with more compression artifacts.

```
ffmpeg -framerate 1 -i frame_%03d.png -c:v libx264 -pix_fmt yuv420p -crf 18 -r 30 high_quality.mp4
ffmpeg -framerate 1 -i frame_%03d.png -c:v libx264 -pix_fmt yuv420p -crf 28 -r 30 small_file.mp4
```

For 5 images at 640x360, CRF 18 produces a 176KB file. CRF 28 produces 61KB, a 2.9x size reduction. For product

screenshots where sharpness matters, stay between 18 and 22. For social media where the platform re-encodes anyway, 23 to 28 is fine.

For a deeper explanation of CRF values and how they affect quality, see the FFmpeg CRF guide.

Image to Video via the FFmpeg Micro API

If you're building this into an app, a content pipeline, or an n8n/Make.com workflow, you don't want to manage an FFmpeg binary on a server. FFmpeg Micro accepts image URLs as inputs and handles the encoding in the cloud.

Basic image-to-video request:

```
curl -X POST https://api.ffmpeg-micro.com/v1/
transcodes \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "inputs": [
      {
        "url": "https://example.com/
frame1.png",
        "options": [
          {"option": "-loop", "argument": "1"},
          {"option": "-framerate", "argument":
"30"},
          {"option": "-t", "argument": "3"}
        ]
      },
      {
        "url": "https://example.com/
```

```
frame2.png",
        "options": [
          {"option": "-loop", "argument": "1"},
          {"option": "-framerate", "argument":
"30"},
          {"option": "-t", "argument": "3"}
        ]
      }
    ],
    "outputFormat": "mp4",
    "options": [
      {"option": "-c:v", "argument":
"libx264"},
      {"option": "-pix_fmt", "argument":
"yuv420p"},
      {"option": "-crf", "argument": "23"}
    ]
  }'
```

The API creates each image as a clip using the per-input options, concatenates them, and applies the output-level encoding options. Poll for completion, then download:

```
curl https://api.ffmpeg-micro.com/v1/
transcodes/JOB_ID \
  -H "Authorization: Bearer YOUR_API_KEY"

curl https://api.ffmpeg-micro.com/v1/
transcodes/JOB_ID/download \
  -H "Authorization: Bearer YOUR_API_KEY"
```

Each input can have different duration and frame rate settings. The API accepts JPEG, PNG, GIF, and WebP images at public HTTP URLs or uploaded via the presigned upload

flow.

Common Pitfalls

Dimensions must be even. H.264 requires both width and height to be divisible by 2. A 641x481 image will fail with `height not divisible by 2`. Fix it by scaling:

```
ffmpeg -framerate 1 -i frame_%03d.png -c:v
libx264 -pix_fmt yuv420p -vf
"scale=trunc(iw/2)*2:trunc(ih/2)*2" -r 30
output.mp4
```

Mixed image sizes break the output. If your images have different dimensions, FFmpeg uses the first image's size and distorts the rest. Normalize everything to a consistent resolution:

```
-vf
"scale=1920:1080:force_original_aspect_ratio=de
(ow-iw)/2:(oh-ih)/2"
```

This scales each image to fit within 1920x1080 while preserving its aspect ratio, then pads with black bars.

Alpha channels in PNGs cause playback failures. PNGs with transparency produce yuv444p output by default, which Chrome, Safari, and most mobile players can't decode. The `-pix_fmt yuv420p` flag fixes this by stripping the alpha channel and converting to a universally compatible format. Don't skip it.

Numbering gaps skip images. If your files jump from `img_003.png` to `img_005.png`, FFmpeg stops at the gap. Either rename to fill the gap, or switch to a glob pattern.

Framerate confusion: input vs output. `-framerate`

(before `-i`) controls how many images FFmpeg reads per second. `-r` (after `-i`) controls the output file's frame rate. They're not the same thing. If you set `-framerate 1 -r 30`, each image displays for 1 second, but the output contains 30 duplicate frames per image for smooth playback.

FAQ

What image formats does FFmpeg support for video creation?

FFmpeg reads JPEG, PNG, BMP, TIFF, WebP, and most standard image formats. JPEG is fastest to decode. PNG preserves sharp edges for screenshots and text. Use PNG for UI mockups and screenshots, JPEG for photographs.

How do I control how long each image appears in the video?

Set `-framerate` before the input. `-framerate 1` means 1 image per second. `-framerate 1/3` means each image displays for 3 seconds. For different durations per image, create individual clips with `-loop 1 -t <seconds>` and concatenate them with the concat demuxer.

Can I add transitions between images?

Yes. FFmpeg's `xfade` filter supports over 40 transition styles including fade, dissolve, wipe, and slide. See the slideshow from images guide for detailed xfade examples with multiple images.

Is there a maximum number of images FFmpeg can process?

No hard limit in FFmpeg itself, but memory usage scales

with image resolution and count. For thousands of large images, process in batches of 100-200 and concatenate the resulting clips. FFmpeg Micro's API accepts up to 10 images per request.

How do I add background audio to an image video?

Add the audio file as a separate input and use `-shortest` to cut when the video ends: `ffmpeg -framerate 1 -i img_%03d.png -i music.mp3 -c:v libx264 -c:a aac -pix_fmt yuv420p -shortest output.mp4`.

Last verified: 2026-05-28 against FFmpeg 3.3.3 and FFmpeg Micro production API.